School ERP, Website and Mobile App

Complete Project Blueprint for
M.D. Children Academy (MDCA), Raisinghnagar

Technical and non-technical guide: requirements, modules, architecture, database, backend, frontend, mobile app, security, reliability, testing, deployment, roadmap and cost.

Model reference: Digii Campus style ERP (student login, parent login, marks, classes, fees, attendance, notifications)
Version 1.0 | October 2026

Contents

1. Read This First 4

1.1 What Digii Campus offers (public information) . . . 4

1.2 What MDCA needs . . . 4

1.3 Goals of the project . . . 4

2. Non-Technical Overview (For School Management) 5

2.1 What the school gets, in plain language . . . 5

2.2 Who uses it . . . 5

2.3 Benefits . . . 5

3. Module-by-Module Feature List 6

4. Role-Wise Screens and Menus 9

4.1 Student / Parent App . . . 9

4.2 Teacher App and Web . . . 9

4.3 Admin Web Panel . . . 9

5. System Architecture 10

5.1 Architectural decision: modular monolith first . . . 10

5.2 Multi-tenancy (if you will sell to other schools) . . . 10

6. Technology Stack (Recommended) 11

7. Database Design 12

7.1 Core tables . . . 12

7.2 Database rules . . . 13

8. Backend Design 14

8.1 Folder / module structure (NestJS example) . . . 14

8.2 API design (REST, versioned /api/v1) . . . 14

8.3 Authentication and authorization . . . 14

8.4 Business rules that must be coded carefully . . . 14

8.5 Background jobs . . . 15

8.6 Error handling and logging . . . 15

9. Frontend Design (Web ERP) 16

9.1 Structure . . . 16

9.2 Key screens to design first . . . 16

9.3 Performance rules . . . 16

10. Public School Website 17

11. Mobile App 18

11.1 Plan . . . 18

11.2 Store requirements . . . 18

11.3 Release process . . . 18

12. Integrations 19

13. Security and Data Protection 20

13.1 Technical controls . . . 20

13.2 Legal and policy (India) . . . 20

13.3 Operational controls . . . 20

14. Making the System Stable (No Crashes) 21

14.1 Design for load . . . 21

14.2 Design for failure . . . 21

14.3 Monitoring and alerts . . . 21

14.4 Load testing before go-live . . . 21

14.5 Recovery targets . . . 21

15. Testing Plan 23

16. DevOps and Deployment 24

16.1 Starting infrastructure (example for one school) . . . 24

17. Project Roadmap 25

17.1 Best time to launch . . . 25

18. Team and Cost Estimate 26

18.1 Team . . . 26

18.2 Indicative cost (rough estimates, not a quotation) . . . 26

19. Data Migration and Go-Live Checklist 27

20. Master Checklist - Every Point to Check 28

20.1 Functional checks . . . 28

20.2 Technical checks . . . 28

20.3 Business and legal checks . . . 28

21. Risks and How to Reduce Them 29

22. Glossary (Simple Words) 30

23. Recommended Next Steps 31

1. Read This First

Important honesty note. The page gdgu.digiicampus.com is a login-protected web app. Without a GD Goenka University username and password, its inside screens cannot be viewed, so no one (including me) can truthfully claim to have checked every screen of it. This blueprint is therefore built from (a) Digii Campus's publicly described modules, (b) features common to leading Indian school ERPs, and (c) what a CBSE school like MDCA actually needs. If you can get a demo login or screen recordings, send them and the module list can be matched screen by screen.

1.1 What Digii Campus offers (public information)

- **Parent management:** parents see child performance, pay fees online, get receipts, view attendance, see data for several children (wards), and approve or reject gate pass requests.
- **Attendance:** day-wise and course-wise attendance, current-term overview, notifications to parents.
- **Exams and results:** exam results visible to students and parents.
- **Notifications:** in-app alerts for important updates.
- **Suite:** ERP, LMS (learning management), HR and process automation in one platform.
- **Branding:** every college gets its own sub-domain (e.g. gdgu.digiicampus.com) with its own login. You can do the same: *erp.mdcarsnr.in*.

1.2 What MDCA needs

MDCA is a co-educational CBSE school (Nursery to Class 12; PCM, PCB, Commerce and Arts streams) at Vijaynagar Road, Raisinghnagar, Rajasthan 335051. A school ERP differs from a college ERP: classes and sections instead of semesters, class teachers, report cards in CBSE format, transport, parents as the main users, and young children's data that must be protected.

1.3 Goals of the project

- One database holding every detail of every student, parent, teacher and staff member.
- Three connected products: **public school website**, **web ERP** (admin, teacher, accountant, principal) and **mobile app** (parent, student, teacher).
- Student login, parent login, marks, class, attendance, fees, notices - all in real time.
- System must stay up during peak load (fee deadlines, result day) - see Section 14.
- School owns the data and can export it at any time.

2. Non-Technical Overview (For School Management)

2.1 What the school gets, in plain language

- **Admission:** parents fill an online form; office verifies; student gets an admission number and login automatically.
- **Daily attendance:** teacher taps present/absent on phone; parent gets a message within minutes if child is absent.
- **Fees:** the system knows each child's fee, discount, sibling concession and due date; parent pays by UPI/card; receipt is generated automatically; the office sees who has not paid.
- **Exams and report cards:** teachers enter marks once; grades, ranks and CBSE-style report cards are produced automatically.
- **Communication:** notices, homework, circulars, holidays and events go to the right class or whole school at once.
- **Transport:** bus routes, stops, drivers, and (optionally) live GPS tracking.
- **Staff:** teacher attendance, leave, salary slips.
- **Principal dashboard:** today's attendance, fees collected, defaulters, results summary on one screen.

2.2 Who uses it

User	What they do	Device
Student	View timetable, homework, marks, attendance, notices, study material	Phone / web
Parent	View child details, pay fees, see attendance and marks, apply leave, approve gate pass, chat with school	Phone (main) / web
Teacher	Mark attendance, enter marks, post homework, apply leave, message parents	Phone + web
Class teacher	Everything above plus class report cards, student remarks	Phone + web
Accountant	Fee structure, collect fees, receipts, dues, expenses, reports	Web
Front office	Enquiry, admission, ID cards, certificates, visitor log, gate pass	Web
Principal / Director	Dashboards, approvals, all reports	Web + phone
Librarian	Books, issue/return, fines	Web
Transport in-charge	Routes, buses, drivers, student transport assignment	Web
Super admin (IT)	Users, roles, settings, backups, audit logs	Web

2.3 Benefits

- Less paperwork and fewer registers; no lost records.
- Fee leakage reduced; transparent receipts; automatic reminders.
- Parents trust and engage more because they see information in real time.
- Admissions improve with a professional website and online enquiry form.
- Management decisions based on live data instead of guesses.

3. Module-by-Module Feature List

Each module lists the features to build. Mark them Must (launch), Should (phase 2) or Could (later).

3.1 Student Information System (SIS) - Must

- Student profile: name, photo, DOB, gender, blood group, Aadhaar (optional, encrypted), category, religion, nationality, mother tongue, address, previous school, medical notes, house, SR number, admission number, PEN number, APAAR ID.
- Parent/guardian details: father, mother, guardian, occupation, income (optional), phone, email, emergency contacts.
- Sibling linking (one parent login sees all children).
- Document upload: birth certificate, TC, marksheet, photos, vaccination record.
- Class and section allocation, roll numbers, promotion to next class, detention, TC issue, alumni records.
- Bulk import from Excel; export to Excel/PDF.

3.2 Admissions and Enquiry (CRM) - Must

- Public enquiry form on the website; enquiry tracking (new, called, visited, admitted, lost).
- Online application with document upload and application fee payment.
- Entrance test / interview scheduling, merit list, seat count per class, waiting list.
- Admission confirmation, auto admission number, auto login creation, welcome SMS/WhatsApp.

3.3 Academic Setup - Must

- Academic year / session (2026-27), classes Nursery to 12, sections, streams (PCM, PCB, Commerce, Arts).
- Subjects, subject groups, electives (Class 11-12), subject-teacher mapping, class-teacher mapping.
- Timetable builder with clash detection (teacher or room double-booked), substitute teacher arrangement.
- Holiday calendar and academic calendar.

3.4 Attendance - Must

- Daily class attendance and period-wise attendance from teacher app (works offline, syncs later).
- Lock attendance after a time limit; admin override with reason (audit logged).
- Absence alert to parent by app push / SMS / WhatsApp.
- Student leave application by parent; approval by class teacher.
- Optional: biometric / RFID / face device integration, bus boarding attendance.
- Reports: daily, monthly, below-75% list, class-wise, student-wise.

3.5 Fee Management - Must

- Fee heads: admission, tuition, transport, hostel, exam, lab, activity, late fee, fine, misc.
- Fee structure per class and stream; installment plans (monthly/quarterly); due dates.
- Concessions: sibling, staff ward, merit, EWS/RTE, manual discount with approval.
- Online payment (UPI, cards, net banking) via a payment gateway with automatic reconciliation; cash/cheque/DD at counter.
- Auto receipt PDF with school logo, receipt number series, duplicate receipt, cancellation with approval.
- Late-fee rules, advance payment, partial payment, refund, carry-forward of dues.
- Reminders (SMS/WhatsApp/app) before and after due date; defaulter list.

- Daily collection report, head-wise report, mode-wise report, day-book, ledger, outstanding report, Excel/PDF export.

3.6 Examination and Results - Must

- CBSE patterns: Class 1-5 and 6-8 (term/periodic tests, notebook, subject enrichment), 9-10 (internal assessment + board pattern), 11-12 (unit tests, practical, theory).
- Exam schedule, admit cards, seating plan, invigilation duty.
- Marks entry by subject teacher with entry window, lock and approval; grace marks; absent/medical flags.
- Grade scale (A1-E2), percentage, rank, co-scholastic grades, attendance on report card, remarks.
- Report card PDF in school format; bulk generation; class result analysis; subject-wise analysis; topper list.
- Parent/student see marks only after school publishes them.

3.7 Homework, Diary and Learning Material - Should

- Daily homework posting with attachments; student submission (photo/PDF); teacher remarks.
- Class diary, circulars, notes, worksheets, video links; simple online test / quiz (later).
- Live class links (Zoom/Meet) shown in the app.

3.8 Communication - Must

- Notice board, circulars, event posts, gallery, birthday wishes.
- Push notifications, SMS and WhatsApp Business messages; templates approved as required by DLT rules.
- Parent-teacher messaging with school-hours control and message moderation; PTM scheduling; feedback and complaint box.

3.9 Transport - Should

- Vehicles (registration, fitness, insurance, permit dates), drivers and conductors, routes and stops, fee by stop.
- Student-to-route assignment; bus attendance; optional live GPS tracking with parent app map and arrival alerts.

3.10 Library - Could

- Book catalogue with barcode, issue/return, fine, member cards, reports.

3.11 Hostel - Could (only if MDCA runs one)

- Rooms, beds, allotment, mess, hostel fee, leave and out-pass.

3.12 HR and Payroll - Should

- Staff profile and documents, joining, contracts, qualifications.
- Staff attendance (app/biometric), leave types and balance, approvals.
- Salary structure, deductions (PF/ESI/TDS where applicable), payslip PDF, salary register.

3.13 Accounts and Expenses - Should

- Income and expense heads, vouchers, vendor list, petty cash, bank book, monthly P&L, export to Tally.

3.14 Front Office and Gate - Should

- Visitor log, gate pass with parent approval (as Digii does), early-leave requests, call log, postal register.
- Certificates: bonafide, character, TC, fee certificate - auto-generated with number and QR verification.

- ID cards with photo and QR/barcode, bulk print.

3.15 Inventory and Store - Could

- Uniform, books, stationery stock; sale to students linked to fee account; purchase orders.

3.16 Reports and Dashboards - Must

- Principal dashboard: attendance today, fee collected today/month, pending dues, admissions funnel, staff absent.
- Standard reports list for every module, filters by class/section/date, export Excel/PDF, scheduled email of daily report.

3.17 Administration and Security - Must

- Users, roles and permissions, school settings, session rollover, audit log, backup/restore, data export, multi-branch ready.

4. Role-Wise Screens and Menus

4.1 Student / Parent App

Screen	Contents
Login	Mobile number + OTP or username/password; child switcher for siblings
Home dashboard	Today's attendance, due fee card, latest notice, homework due, next exam
Profile	Student and parent details; request correction
Attendance	Calendar view (green present, red absent), monthly %
Fees	Due list, pay now, receipts, history, download PDF
Marks and results	Exam-wise marks, grade, rank, report card PDF
Timetable	Day and week view
Homework and material	List, details, attachments, submit
Notices and events	Feed with read/unread status
Leave and gate pass	Apply leave; approve/reject gate pass request
Transport	Route, stop, bus number, live map
Messages	Chat with class teacher (controlled)
Settings	Language (English/Hindi), notification choices, logout, delete-account request

4.2 Teacher App and Web

- Today's classes, mark attendance (one-tap list), enter marks, post homework and diary, apply leave, view timetable, message parents, see my students, view salary slip.

4.3 Admin Web Panel

- Dashboard; Students; Admissions; Academics; Attendance; Fees; Exams; Communication; Transport; Library; HR; Accounts; Front Office; Reports; Settings; Audit log; Backup.

5. System Architecture

The system has one backend serving three front-ends. All three talk to the backend through a secure REST API over HTTPS.

Layer	Description
Clients	(1) Public website, (2) Web ERP for staff, (3) Mobile app for parents/students/teachers
Edge	DNS + CDN + firewall (Cloudflare) + SSL, rate limiting
Load balancer	Nginx or cloud load balancer in front of 2+ application servers
Application	Backend API (stateless) so more servers can be added when traffic grows
Background workers	Queue workers for SMS/WhatsApp, PDF receipts, report cards, bulk imports, reminders
Database	PostgreSQL primary + read replica + automatic backups
Cache	Redis for sessions, rate limits, dashboard cache, queue
File storage	Object storage (S3-compatible) for photos, documents, PDFs; served via signed URLs
Third-party services	Payment gateway, SMS/WhatsApp provider, email, push (Firebase), maps, biometric devices
Observability	Logs, metrics, alerts, uptime monitor, error tracking

5.1 Architectural decision: modular monolith first

For a single school (even a few thousand users) start with a **modular monolith**: one backend codebase divided into clean modules (students, fees, exams...). It is far simpler to build, test, deploy and keep stable than microservices. Because it is modular and stateless, it can later be split or scaled if you sell it to other schools.

5.2 Multi-tenancy (if you will sell to other schools)

- Add a **school_id** to every table from day one. It costs nothing now and avoids a rewrite later.
- Each school gets its own sub-domain (e.g. mdca.yourerp.in) and branding; data is isolated by school_id with row-level checks.
- For MDCA only, you can run single-school mode with the same code.

6. Technology Stack (Recommended)

Part	Recommended	Why / alternative
Backend language and framework	Node.js with NestJS (TypeScript)	Structured modules, strong typing, huge ecosystem. Alternatives: Laravel (PHP) for fastest delivery and cheap hosting; Django (Python)
Database	PostgreSQL 16	Reliable, relational, ideal for fees and marks. MySQL is acceptable
ORM / migrations	Prisma or TypeORM	Versioned schema migrations
Cache and queue	Redis + BullMQ	Background jobs, rate limit
Web ERP frontend	React + Next.js or Vite, TypeScript, Tailwind CSS, a component library (MUI/shadcn)	Fast, huge talent pool
Public website	Next.js (server rendered) or WordPress for easy content editing	SEO matters for admissions. Content managed from ERP or a CMS
Mobile app	Flutter (one codebase Android + iOS) or React Native	Cheaper than two native apps. Android first, as most parents use Android
Auth	JWT access + refresh tokens, OTP login, role-based access control (RBAC)	Passwords hashed with Argon2/bcrypt
File storage	S3-compatible (AWS S3, Cloudflare R2, DigitalOcean Spaces)	Never store files on the app server disk
Payments	Razorpay / PayU / Cashfree / ICICI Eazypay	UPI + cards + net banking; webhooks for confirmation
Messaging	MSG91 / Gupshup / Twilio, WhatsApp Business API, Firebase Cloud Messaging	DLT-registered SMS templates for India
PDF generation	Puppeteer or PDFKit	Receipts, report cards, ID cards
Hosting	AWS (Mumbai region) / DigitalOcean / Hetzner + Cloudflare	Indian region for latency and data residency
CI/CD	GitHub Actions + Docker	Automatic tests and deploy
Monitoring	Sentry, Grafana/Prometheus or Better Stack, UptimeRobot	Alerts to phone when something breaks

Which language should YOU choose? Pick the one your team knows best. If you are solo and want speed, Laravel + MySQL + Flutter is the fastest path. If you want a modern scalable product for many schools, NestJS + PostgreSQL + React + Flutter is the stronger long-term choice.

7. Database Design

Relational schema (PostgreSQL). Every table has: id (UUID), school_id, created_at, updated_at, created_by, and deleted_at (soft delete). Money is stored as integer paise, never floating point.

7.1 Core tables

Table	Key columns
schools	name, code, address, phone, email, logo_url, board_affiliation_no, settings_json
academic_years	name (2026-27), start_date, end_date, is_current
users	mobile, email, password_hash, role_id, status, last_login_at, mfa_enabled
roles / permissions / role_permissions	role name; permission key e.g. fees.collect, marks.edit; mapping
students	admission_no, sr_no, first_name, last_name, dob, gender, blood_group, photo_url, category, aadhaar_enc, address, status, admission_date, user_id
guardians	name, relation, mobile, email, occupation, user_id
student_guardians	student_id, guardian_id, is_primary, can_pay_fees
student_documents	student_id, doc_type, file_url, verified_by
classes	name (Class 1..12), display_order
sections	class_id, name (A,B), stream (PCM/PCB/Commerce/Arts), class_teacher_id, capacity
student_enrollments	student_id, academic_year_id, section_id, roll_no, status (active/promoted/TC)
subjects	name, code, type (scholastic/co-scholastic), is_elective
section_subjects	section_id, subject_id, teacher_id
staff	employee_no, name, designation, department, joining_date, qualification, salary_structure_id, user_id
timetable_slots	section_id, day, period_no, subject_id, teacher_id, room
attendance_sessions	section_id, date, period_no, taken_by, locked_at
attendance_records	session_id, student_id, status (P/A/L/HD), remark
leave_requests	student_id, from_date, to_date, reason, status, approved_by
fee_heads	name, type, is_refundable
fee_structures	academic_year_id, class_id, fee_head_id, amount, frequency, due_day
student_fee_ledger	student_id, fee_head_id, period, amount_due, concession, amount_paid, status, due_date
concessions	student_id, type, percent_or_amount, approved_by, reason
payments	student_id, receipt_no, total_amount, mode, gateway_txn_id, status, paid_at, collected_by
payment_items	payment_id, ledger_id, amount
exams	academic_year_id, name, type, start_date, end_date, weightage, is_published
exam_subjects	exam_id, class_id, subject_id, max_marks, pass_marks, exam_date
marks	exam_subject_id, student_id, marks_obtained, grade, is_absent, entered_by, locked
report_cards	student_id, exam_set, pdf_url, generated_at, published
homework	section_id, subject_id, title, description, due_date, attachments, posted_by
homework_submissions	homework_id, student_id, file_url, submitted_at, remark
notices	title, body, audience_type (all/class/section/role), attachments, published_at
notifications	user_id, channel, title, body, status, sent_at, read_at

Table	Key columns
enquiries / applications	child name, parent mobile, class sought, source, status, follow_ups, documents
vehicles / routes / stops / student_transport	reg_no, capacity, driver_id; route name; stop fee; student-stop mapping
gate_passes	student_id, requested_by, reason, out_time, status, parent_decision, approved_by
library_books / library_issues	isbn, title, copies; member, issue_date, due_date, fine
salary_structures / payslips	components, deductions; month, gross, net, pdf_url
audit_logs	user_id, action, entity, entity_id, old_value, new_value, ip, timestamp
files	owner_type, owner_id, storage_key, mime, size, uploaded_by

7.2 Database rules

- Foreign keys and unique constraints (e.g. unique admission_no per school, unique receipt_no per year).
- Indexes on: student_id, section_id, date, status, mobile, admission_no, receipt_no.
- Receipt numbers generated inside a database transaction using a sequence so duplicates are impossible.
- Payments are append-only: corrections are made by cancel + new entry, never by editing history.
- Soft delete for students and staff; hard delete only by super admin under policy.
- Migrations are versioned and tested on a copy of production before release.

8. Backend Design

8.1 Folder / module structure (NestJS example)

- src/modules/auth, users, roles, students, guardians, admissions, academics, attendance, fees, payments, exams, homework, notices, notifications, transport, library, hr, accounts, frontoffice, reports, files, audit, settings
- Each module: controller (routes), service (business logic), repository (database), dto (validation), tests.

8.2 API design (REST, versioned /api/v1)

Method	Endpoint	Purpose
POST	/auth/login, /auth/otp/request, /auth/otp/verify, /auth/refresh, /auth/logout	Login and tokens
GET/POST/PATCH	/students, /students/:id, /students/:id/documents	Student CRUD, search, filters
GET	/me/children	Parent: list of own children
POST	/attendance/sessions, /attendance/sessions/:id/records	Mark attendance (bulk)
GET	/students/:id/attendance?month=	Attendance calendar
GET/POST	/fees/structures, /fees/ledger/:studentId	Fee setup, dues
POST	/payments/initiate, /payments/webhook, /payments/counter	Online and counter payment
GET	/payments/:id/receipt.pdf	Receipt download
POST/GET	/exams, /exams/:id/marks, /students/:id/report-card	Exams and marks
POST/GET	/homework, /notices, /notifications	Communication
POST/PATCH	/leave-requests, /gate-passes/:id/decision	Leave and gate pass
GET	/reports/fees/daily, /reports/attendance/monthly, /dashboard/principal	Reports
GET	/audit-logs	Audit trail (admin)

8.3 Authentication and authorization

- **Login:** staff use email/username + password with optional 2-factor; parents and students use mobile OTP or password.
- **Tokens:** short-lived access token (15 min) + refresh token (30 days) stored securely; revoke on logout or password change.
- **RBAC:** permissions like students.view, fees.collect, marks.edit. Roles are bundles of permissions; the API checks permission on every route.
- **Data scoping:** a parent can only fetch their own children; a teacher only their own classes. Enforced in the service layer, not only in the UI.
- **Rate limiting** on login and OTP to stop brute-force and SMS abuse.

8.4 Business rules that must be coded carefully

- Fee calculation: base fee - concession + late fee + transport; order of applying advance payments; rounding.
- Payment confirmation only from the gateway **webhook** (server-to-server), never from the app's success screen alone; webhook is idempotent.
- Marks entry window and lock; changes after lock need approval and are audit logged.
- Promotion logic: pass/fail/detain to next section and year; carry-forward of dues.

- Attendance lock time and correction approval.
- Report-card computation exactly as per CBSE scheme chosen by the school (confirm with the principal).

8.5 Background jobs

- Send SMS/WhatsApp/push in batches with retry and failure log.
- Generate receipts and bulk report-card PDFs without blocking users.
- Nightly: fee reminders, late-fee posting, birthday wishes, backup verification, daily summary email to principal.
- Bulk import of students from Excel with row-level error report.

8.6 Error handling and logging

- Consistent JSON error format (code, message, field errors); never expose stack traces to users.
- Structured logs with a request id; errors sent to Sentry; alert if error rate crosses a threshold.

9. Frontend Design (Web ERP)

9.1 Structure

- Layout: sidebar by module, top bar with academic year, search, notifications, profile.
- Reusable components: data table (search, filter, sort, pagination, export), form, modal, date picker, file uploader, charts.
- State: React Query for server data; small global store for user and settings.
- Forms validated in the browser and again in the backend.
- Responsive: works on laptop, tablet and phone; print-friendly receipts and reports.
- Accessibility: keyboard navigation, readable fonts, good contrast. Languages: English + Hindi.

9.2 Key screens to design first

- Login, dashboard, student list, student 360-degree profile (tabs: details, guardians, attendance, fees, marks, documents, timeline)
- Admission form, fee collection counter screen (fast, keyboard-friendly), fee structure setup, defaulters list
- Attendance register, marks entry grid (spreadsheet-like), report-card preview, notice composer, timetable grid, reports center.

9.3 Performance rules

- Server-side pagination for every large list; never load all students at once.
- Lazy load pages and charts; compress images; cache static assets on CDN.
- Target: page interactive under 3 seconds on a normal 4G connection.

10. Public School Website

The website is the school's face and the main driver of admissions. It is separate from the ERP login but connected to it.

Page	Content
Home	Hero banner, admission open banner, principal message, highlights, results, news ticker, gallery preview
About	History (established 2008/2009), vision and mission, management, CBSE affiliation number, infrastructure
Academics	Classes Nursery-12, streams PCM/PCB/Commerce/Arts, curriculum, faculty
Admissions	Process, fee structure, dates, online enquiry/application form (feeds the ERP CRM)
Mandatory Public Disclosure	CBSE-required disclosure documents, affiliation, fee, staff, infrastructure details
Facilities	Labs, library, transport, sports, smart classes
Gallery and events	Photos, videos, annual day, sports day
Results and toppers	Board results year-wise
Notices and circulars	Pulled live from the ERP
Contact	Address (RSNR Vijaynagar Road, Raisinghnagar, 335051), phones, email, map, enquiry form
Login	Buttons for Parent/Student login, Staff login (to ERP domain), Download app

- SEO: fast loading, mobile friendly, proper titles and meta tags, Google Business Profile, schema markup, sitemap.
- Content editable by a school staff member without a developer (CMS pages and gallery upload).
- Domain such as mdcarsnr.in; SSL; separate sub-domains: *www* (website), *erp* (staff), *app/api* (backend).
- Link Instagram (@mdcarsnr) and Facebook feeds.

11. Mobile App

11.1 Plan

- One app with role-based login (parent, student, teacher) or two apps (Parent/Student and Teacher). One app is easier to maintain.
- Android first (Play Store), iOS second (App Store). A school's app name, icon and colors should match MDCA's identity.
- Offline support: teacher attendance and homework drafts save locally and sync when the network returns.
- Push notifications via Firebase Cloud Messaging; deep links open the correct screen (e.g. fee due).
- Biometric/PIN app lock; secure token storage (Keychain/Keystore); certificate pinning for sensitive screens.
- Online payment via gateway SDK or in-app webview; receipt available instantly after webhook confirmation.
- App size small (under 30 MB) and works on low-end Android phones, because many parents use them.

11.2 Store requirements

- Google Play developer account (one-time fee), Apple Developer account (yearly fee).
- Privacy policy URL, data safety form, account deletion option (required by both stores).
- Because the app has children's data, describe clearly that it is used by parents/guardians and school staff.
- Screenshots, app description in English and Hindi, support email.

11.3 Release process

- Internal testing -> closed beta with 20 parents/teachers -> staged rollout (10%, 50%, 100%).
- Force-update flag from the backend if a critical bug or security fix needs everybody to upgrade.

12. Integrations

Integration	Purpose	Notes
Payment gateway	Online fees	UPI/cards/net banking; webhook + reconciliation report; settlement to school bank account
SMS (DLT registered)	OTP, absence, fee alerts	Register entity, header and templates under TRAI DLT; keep fallback provider
WhatsApp Business API	Notices, receipts, reminders	Pre-approved templates; parent opt-in
Firebase Cloud Messaging	Push notifications	Free; cheapest channel
Email (SES/Postmark)	Reports, receipts, password reset	SPF/DKIM configured
Biometric / RFID / face devices	Student and staff attendance	Via device SDK/ADMS push or CSV sync service
GPS trackers	Bus live tracking	Vendor API or MQTT; map via Google Maps or OpenStreetMap
Tally / accounting	Export vouchers	XML/Excel export
Government / CBSE	Student data formats (e.g. APAAR, UDISE+ style exports)	Provide export templates; verify current formats before use
Google Workspace / Zoom	Online classes	Link only in phase 1

13. Security and Data Protection

A school ERP holds identity data, phone numbers, photos and payment history of children. A leak would be serious, so security is a design requirement, not an extra.

13.1 Technical controls

- HTTPS everywhere (TLS 1.2+), HSTS; secure cookies; strict CORS.
- Passwords hashed with Argon2/bcrypt; enforce strong passwords for staff; 2-factor for admin and accountant roles.
- Role-based access plus data scoping; principle of least privilege; separate admin domain.
- Protection against OWASP Top 10: SQL injection (parameterized queries), XSS (output encoding, CSP), CSRF, broken access control, insecure file upload (type/size check, virus scan, store outside web root), SSRF.
- Sensitive fields (Aadhaar, bank details) encrypted at rest; database encryption and encrypted backups.
- Files served via short-lived signed URLs; parents cannot guess another child's document link.
- Audit log of every change to fees, marks, attendance and user permissions.
- Secrets in a secret manager, never in the code repository; dependency scanning and updates monthly.
- Web application firewall, DDoS protection and bot protection through Cloudflare.
- Annual penetration test or at least an independent security review before launch.

13.2 Legal and policy (India)

- Digital Personal Data Protection Act, 2023: children's data needs verifiable parental consent, no tracking or targeted advertising to children, purpose limitation and the ability to correct/erase data. Take legal advice and check the current rules before launch.
- Publish a Privacy Policy and Terms of Use; keep a consent record at admission.
- Data retention policy (how long old student data is kept) and a breach response plan.
- Keep Indian student data on servers in India where possible.
- Do not claim to be a lawyer-approved compliance product unless a lawyer has reviewed it.

13.3 Operational controls

- Staff accounts created by admin only; immediate disable when staff leave.
- Never share logins; each action tied to a person.
- Phishing and password training for staff; phone lock on teacher devices.

14. Making the System Stable (No Crashes)

No software can promise zero downtime, but a well-built system can be reliable enough that users rarely notice a problem. A sensible target is **99.9% uptime** (about 8.7 hours of downtime per year), with the worst times being fee deadlines and result day.

14.1 Design for load

- Estimate load: 1,500 students means about 3,000-4,500 parent/student accounts. Peak: 500-1,000 people opening the app in the same hour on fee due date or results day. This is modest; a properly built system on 2 small servers handles it.
- Stateless backend + load balancer + at least 2 app servers, so one failure does not stop service.
- Database connection pooling (PgBouncer), indexed queries, no query inside loops (N+1), pagination everywhere.
- Cache dashboards, class lists and notices in Redis; cache static files on CDN.
- Heavy work (bulk SMS, PDFs, imports) always goes to the background queue.
- Results and report cards are pre-generated and published, not computed when parents open them.
- Schedule fee reminders in staggered batches instead of sending to everyone at once.

14.2 Design for failure

- Health checks and auto-restart (Docker/PM2/systemd or managed container service).
- Database: daily full backup + point-in-time recovery (WAL archiving); read replica; backups stored in a different region.
- **Restore drills** every quarter - a backup that was never restored is not a backup.
- Timeouts, retries with backoff and circuit breakers for payment/SMS providers; second SMS provider as fallback.
- Idempotency keys on payment and form submissions so double taps never create double records.
- Graceful degradation: if SMS fails, push still works; if WhatsApp fails, SMS is used; app caches last data to show offline.
- Zero-downtime deployments (rolling or blue-green); database migrations that are backward compatible.
- Maintenance window announced in advance (e.g. Sunday 2 AM).

14.3 Monitoring and alerts

- Uptime monitor pinging website, API and login every minute with SMS/WhatsApp alert to the developer.
- Dashboards: CPU, memory, disk, DB connections, slow queries, queue length, error rate, payment success rate.
- Error tracking (Sentry) with alerts; weekly review of top errors.
- Status page for the school ('All systems normal').

14.4 Load testing before go-live

- Use k6 or JMeter to simulate 2-3 times the expected peak (e.g. 2,000 users) on login, fee payment, attendance submission and result view.
- Fix the slowest queries; repeat the test; keep the report.

14.5 Recovery targets

Metric	Target
RPO (maximum data loss)	5 minutes (point-in-time recovery)
RTO (time to restore)	Under 1 hour for full outage; under 5 minutes for single server failure
Uptime	99.9%
API response (95th percentile)	Under 500 ms for normal requests

15. Testing Plan

Test type	What is checked	Tools
Unit tests	Fee calculation, grade logic, promotion, permissions	Jest / PHPUnit / PyTest
Integration tests	API + database, payment webhook, notification queue	Supertest, Testcontainers
End-to-end tests	Full flows: admission to fee payment to report card	Playwright / Cypress
Mobile tests	Real low-end Android devices, poor network, offline sync	Flutter test, Firebase Test Lab
Load tests	Peak days	k6, JMeter
Security tests	OWASP checks, access-control tests (parent A cannot read child B)	ZAP, manual review
User acceptance (UAT)	Real teachers, accountant, office, 20 parents using test data	Checklist + feedback form
Data migration tests	Old register/Excel data imported correctly, totals match	Reconciliation scripts
Regression	Run automated suite before every release	CI pipeline

- Rule: money and marks code needs the highest test coverage; every bug found in production gets a test written for it.

16. DevOps and Deployment

- **Environments:** local, staging (copy of production with fake data), production.
- **Code:** Git (GitHub/GitLab), main branch protected, pull-request reviews, semantic versions.
- **CI/CD:** every push runs lint, tests, security scan; merge to main deploys to staging automatically; production deploy needs one click approval.
- **Containers:** Docker images; start simple with Docker Compose on one or two servers, move to managed containers/Kubernetes only if needed.
- **Infrastructure as code:** Terraform/Ansible scripts so the whole setup can be rebuilt in an hour.
- **Config:** environment variables; separate keys for test and live payment gateway.
- **Documentation:** API docs (Swagger/OpenAPI), database diagram, runbook ('if X breaks, do Y'), user manuals, training videos.

16.1 Starting infrastructure (example for one school)

Item	Suggested spec
App servers	2 x (2 vCPU, 4 GB RAM)
Database	Managed PostgreSQL, 2 vCPU, 4-8 GB RAM, 100 GB storage, daily backup + PITR
Redis	Small managed instance (1 GB)
Storage	S3-compatible bucket, 100-200 GB
CDN + WAF	Cloudflare (free or pro plan)
Domain + SSL	mdcarsnr.in (or .com) with free SSL

17. Project Roadmap

Phase	Duration (team of 4-5)	Deliverables
0. Discovery	2-3 weeks	Interviews with principal, office, accountant, teachers; collect real forms (admission form, fee receipt, report card, ID card); finalize modules; clickable design
1. Foundation	3-4 weeks	Architecture, database, auth, roles, school settings, student + guardian + class modules, CI/CD, staging
2. Core MVP	8-10 weeks	Admissions, attendance, fees + online payment, exams + report cards, notices, parent/teacher app v1, public website
3. Pilot	3-4 weeks	One or two classes use it live; fix issues; load test; security review; staff training
4. Full launch	2 weeks	All classes, data migration complete, parallel run with paper for 2-4 weeks, go-live
5. Phase 2 modules	8-12 weeks	Homework, transport + GPS, library, HR/payroll, accounts, inventory, biometric, WhatsApp, analytics
6. Support	Ongoing	Monitoring, monthly updates, backup drills, yearly session rollover, new CBSE changes

Total to a stable full launch: roughly **6-8 months** for a small team; a solo developer should plan 10-14 months and ship the MVP (attendance, fees, marks, notices, parent app) first.

17.1 Best time to launch

- Go live at the start of an academic session or at the start of a quarter, not in the middle of exams.
- Run old and new systems in parallel for one month so nothing is lost.

18. Team and Cost Estimate

18.1 Team

Role	Count	Responsibility
Product owner / business analyst	1	Requirements, school liaison, priorities, UAT
UI/UX designer	1 (part-time ok)	Design system, screens, app and website design
Backend developer	1-2	API, database, payments, integrations
Frontend developer	1	Web ERP and website
Mobile developer	1	Flutter app
QA tester	1 (part-time ok)	Test plans, automation
DevOps / security	part-time	Servers, backups, monitoring, security review

18.2 Indicative cost (rough estimates, not a quotation)

Costs vary widely by city and team. The figures below are only planning ranges in Indian rupees; get 2-3 quotes before committing.

Item	One-time (Rs.)	Yearly (Rs.)
Build by a small agency/freelance team (MVP)	8,00,000 - 25,00,000	-
Build by a solo/in-house developer	Salary-based; 10-14 months	-
Cloud hosting, database, storage, CDN	-	60,000 - 2,50,000
SMS / WhatsApp messages	-	30,000 - 1,50,000 (usage based)
Payment gateway fees	-	Per transaction, about 0-2% (negotiate; UPI is often cheaper)
Domain, SSL extras, email	-	3,000 - 15,000
Google Play + Apple developer accounts	Small one-time Play fee	Apple yearly fee
Maintenance and support	-	15-25% of build cost per year
Security review / penetration test	50,000 - 2,00,000	Yearly repeat

Alternative: if budget is tight, buy a ready-made ERP (Digii and many Indian vendors charge per-student yearly fees) or buy a white-label source code product (some vendors sell PHP/Laravel + MySQL + React school ERPs with source code) and customize it. Build from scratch only if you want to own the product and sell it to other schools.

19. Data Migration and Go-Live Checklist

- Collect existing data: admission registers, Excel sheets, fee ledgers, marks of last year, staff list.
- Clean data: duplicate students, missing mobile numbers, wrong classes, spelling; fix before import.
- Import in a staging environment; reconcile totals (number of students per class, total fee dues) with the school's records; sign-off by the accountant.
- Create parent logins; confirm each parent's mobile number is correct (OTP depends on it).
- Train staff: 2 sessions for teachers, 2 for office/accounts, 1 for the principal; give short Hindi/English video guides.
- Parent communication: circular, WhatsApp message, QR code poster at the gate, help desk for the first two weeks.
- Helpline: a phone number and WhatsApp number for support; response-time promise.
- Rollback plan: if something severe happens in week 1, switch back to paper for that function while fixing.

20. Master Checklist - Every Point to Check

20.1 Functional checks

- [ ] Student can log in, see profile, timetable, attendance, marks, fees, notices
- [ ] Parent with 2 children can switch between them and pay fees for each
- [ ] Teacher can mark attendance offline and it syncs
- [ ] Absent student triggers a parent alert
- [ ] Fee: concessions, installments, late fee, partial payment, refund, receipt duplication handled
- [ ] Payment gateway failure, double click, and webhook delay do not create wrong balance
- [ ] Marks entry locks after deadline; report card matches CBSE format; ranks correct
- [ ] Promotion to next class and session rollover work; old year data stays read-only
- [ ] Notices reach the right audience only
- [ ] Gate pass flow: request, parent approve/reject, guard sees decision
- [ ] Certificates have unique numbers and QR verification
- [ ] Every report exports correctly to Excel and PDF

20.2 Technical checks

- [ ] All endpoints validated and permission protected; access-control tests pass
- [ ] No N+1 queries; slow query log empty on test
- [ ] Backups run daily and a restore test has succeeded
- [ ] Load test passed at 2-3x expected peak
- [ ] Monitoring and alerts reach a real phone
- [ ] Security scan and penetration test issues fixed
- [ ] Secrets rotated; no keys in code; production gateway keys separate
- [ ] CDN, SSL, HSTS, firewall configured
- [ ] Logs do not contain passwords, OTPs or full card/Aadhaar numbers

20.3 Business and legal checks

- [ ] Privacy policy, terms, parental consent captured
- [ ] DLT templates and WhatsApp templates approved
- [ ] Payment gateway KYC completed; settlement account verified
- [ ] CBSE mandatory disclosure page published
- [ ] Data ownership and exit terms in the developer contract (source code, database export, documentation handover)
- [ ] Support and maintenance agreement signed

21. Risks and How to Reduce Them

Risk	Impact	Mitigation
Scope creep ('add this also')	Delay and cost overrun	Fixed MVP list; change request process; phases
Staff resistance to new system	Low adoption	Involve teachers early, simple screens, training, parallel run
Parents without smartphones or weak internet	Parents miss information	SMS fallback, low-bandwidth app, help desk at school
Payment errors	Financial loss and disputes	Webhook confirmation, idempotency, daily reconciliation
Data breach	Legal and reputation damage	Section 13 controls, audit logs, breach plan
Server outage on result/fee day	Parent anger	Load balancing, caching, load tests, status page
Key developer leaves	Project stalls	Documentation, code reviews, second developer, code in school's/owner's repository
Wrong report-card rules	Embarrassing results	Principal sign-off on sample report cards before launch
Vendor lock-in	Cannot change provider	Standard tech, data export, source code ownership

22. Glossary (Simple Words)

Term	Meaning
ERP	Enterprise Resource Planning - one software that runs all school operations
Backend	The hidden part (server and database) that stores data and applies rules
Frontend	What people see and click: website, web panel, mobile app
API	The messenger that lets the app and website talk to the backend
Database	Organised storage for all records
RBAC	Role-Based Access Control - what each role is allowed to do
Webhook	Automatic message from the payment company telling us a payment succeeded
OTP	One-time password sent to a mobile number
CDN	Network that serves files from nearby servers for speed
Load balancer	Splits traffic across several servers
Uptime	Percentage of time the system is working
Staging	A practice copy of the system used for testing
DLT	India's SMS registration system for commercial messages
PITR	Point-in-time recovery - restore the database to any minute
MVP	Minimum viable product - the first useful version

23. Recommended Next Steps

- **Week 1:** Meet principal, accountant and office staff; collect real forms; confirm CBSE report card format and fee rules.
- **Week 1-2:** Ask Digii (or a similar vendor) for a demo login to compare screens; decide: build, buy, or customize.
- **Week 2-3:** Finalize MVP list, choose tech stack and team, register domain, start DLT and payment gateway KYC (these take time).
- **Week 3-4:** Design wireframes for the 10 key screens; get principal approval.
- **Then:** Follow the roadmap in Section 17; keep a live demo for the school every two weeks.

I can continue with any of these as a next deliverable: a detailed database ER diagram with full column types, a feature-by-feature Digii comparison once you have login access, the full OpenAPI specification, screen wireframes, or starter code for the backend and mobile app.